Upravljanje testiranjem



Autor: Goran Savić

Upravljanje testiranjem

- Odnosi se na upravljanje procesom testiranja
- Uključuje planiranje, implementaciju i nadzor testiranja softvera
- Osim što vodi računa o tehničkoj realizaciji, upravlja i organizacijom ljudskih resursa koji učestvuju u testiranju
  - test tim

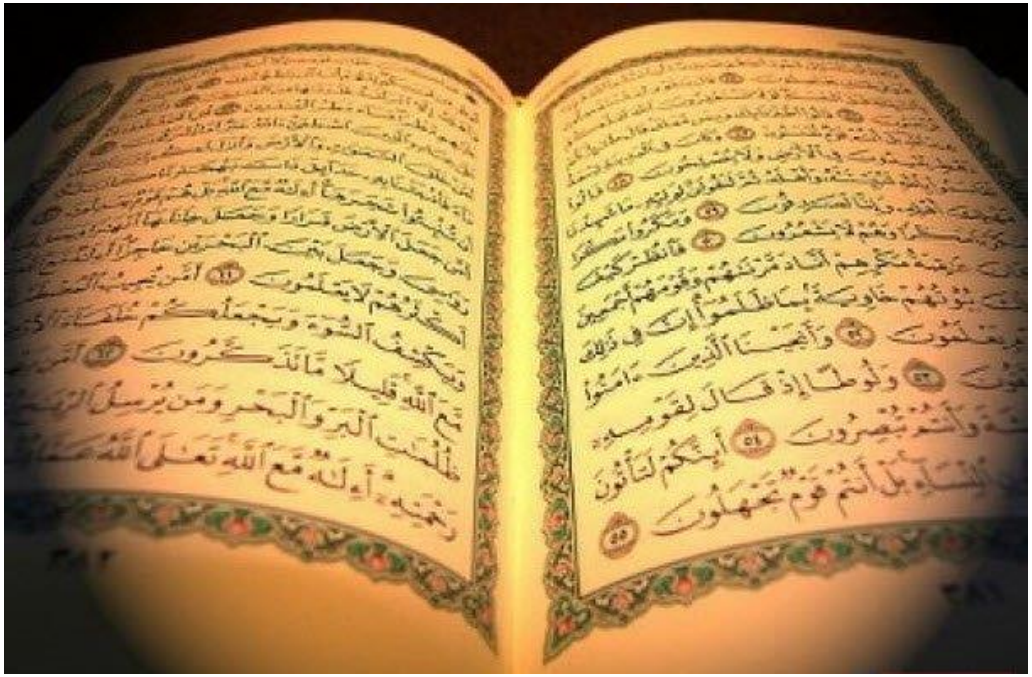
Organizacija test tima



Izbor test tima

- U toku celog procesa razvoja softvera potrebne su i aktivnosti koji se odnose na testiranje
- Neophodno je izabrati koji učesnici projekta će vršiti testiranje
- Dve mogućnosti
  - programeri koji razvijaju softver vrše i testiranje
    - ljudi teže identifikuju sopstvene greške
  - posebni članovi tima vrše testiranje
    - nezavisno testiranje od razvoja softvera

Nezavisno testiranje



A Allah nije ni o kome ovisan i blag je.
(Kuran, 43, 2)

Nezavisno testiranje

- Prednosti
  - nezavisni testeri su nepristrasni
    - nalaze više nedostataka nego programeri
    - nalaze drugačije nedostatke nego programeri
  - programer implicitno podrazumeva određeno ponašanje programa
    - tester ne polazi od istih pretpostavki
    - što ne znači da nema svoje pretpostavke i ponašanje koje on implicitno podrazumeva
    - često je implicitno razumevanje testera bliže razumevanju krajnjeg korisnika

Nezavisno testiranje

- Mane
  - manje komunikacije između članova koji razvijaju i onih koji testiraju sistem
  - zahteva dodatne resurse koji mogu da postanu „usko grlo“ razvoja sistema ako ih je premalo
  - programeri mogu izgubiti osećaj odgovornosti za kvalitet softvera koji razvijaju
    - računaju da je svakako tu tester koji će pronaći nedostatke

Modeli nezavisnog testiranja

1. Testiranje vrši programerski tim, ali niko ne testira svoj kod
2. U okviru razvojnog tima postoji određen broj testera
   - oni vrše sav posao testiranja
3. Postoje nezavisni test timovi zaduženi za testiranje svih projekata
4. Za svaki tip testiranja postoje specijalizovani testeri
   - npr. neko testira samo performanse, neko samo bezbednost, ...
5. Eksterni tim (organizacija) vrši testiranje

- Izbor modela zavisi i od nivoa testiranja

Modeli jediničnog testiranja

- U praksi često jedinične testove vrše sami programeri nad sopstvenim kodom
  - najefikasnije jer poznaju svoj kod
    - posebno važi za *white box* testiranje
  - uz sve mane odsustva nezavisnog testiranja
- Ako se uvodi nezavisno testiranje
  - model 1 u kojem programeri testiraju tuđi kod je prihvatljiv
  - takođe i model 2 sa posebnim testerima u okviru tima

Modeli integracionog testiranja

- Ako se testira integracija komponenti razvijanih u okviru istog tima mogu se koristiti modeli 1 i 2 u kojima u testiranju učestvuju članovi razvojnog tima
- Ako se integrišu komponente različitih timova, potrebni su testeri izvan razvojnih timova
  - modeli 3 i 4, eventualno 5

Modeli sistemskog testiranja

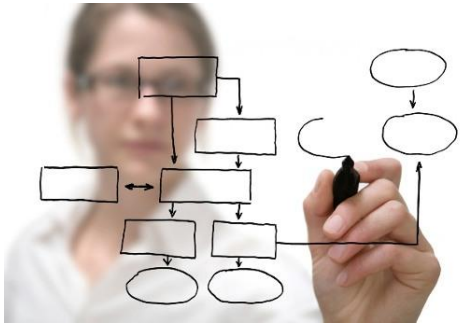
- Kod sistemskog testiranja finalni proizvod mora biti sagledan iz ugla krajnjeg korisnika
- Neophodno je odvojiti testiranje od razvojnog tima
- Modeli 3, 4 i 5 su primereni

Uloge u test timu



- Test menadžer
  - vodi proces testiranja
  - stručan za poslove planiranja i upravljanje testiranjem
  - određuje test strategiju i kreira test plan
  - obezbeđuje resurse za testiranje
    - ljudske i materijalne
  - bira alate i okruženje za testiranje
    - i softversko i hardversko
  - organizuje obuku učesnika za alate i tehnologije potrebne za testiranje
  - obezbeđuje podršku za testiranje
    - *bug-tracking* alati, dokumentacija, ...
  - uvodi i ocenjuje metriku za procenu intenziteta testiranja
  - prilagođava test plan dinamici razvoja i testiranja sistema
  - upravlja test izveštajima i podnosi ih rukovodiocima

Uloge u test timu



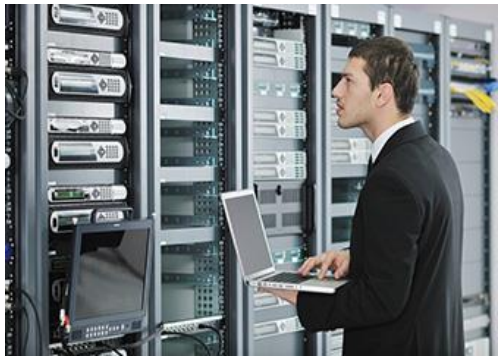
- Test analitičar (dizajner)
  - zadužen za kreiranje testova
  - poznaje softversko inženjerstvo, tehnike testiranja i kreiranje formalnih specifikacija
  - analizira zahteve i specifikaciju projekta
  - na osnovu toga dizajnira test slučajeve
  - obezbeđuje test podatke

Uloge u test timu



- Test automator
  - automatizuje izvršavanje test slučajeva
  - poznaje
    - programiranje,
    - testiranje,
    - test alate,
    - sistemske alate

Uloge u test timu



- Test administrator
  - zadužen za softversku i hardversku infrastrukturu za testiranje
  - ima znanje sistemskog administratora
  - administrira server i test alate
  - podešava okruženje za testiranje
  - obezbeđuje dostupnost okruženja
  - automatizuje izvršavanje testova
  - obezbeđuje skladištenje podataka koji se koriste u testiranju ili su rezultat testiranja

Uloge u test timu



- Tester
  - zadužen za izvršavanje testova i prijavu nedostataka
  - ima generalna znanja iz oblasti IT, znanje testiranja, zna da koristi test alate i poznaje test objekat
  - analizira test plan i test slučajeve
  - dokumentuje rezultate testova
  - koristi test alate za izvršavanje testova i nadzor nad testiranjem

Uloge u test timu



- U praksi često ista osoba preuzima više uloga
- Npr. ista osoba dizajnira i automatizuje test slučajeve
- U manjim timovima, osoba koja upravlja testiranjem učestvuje i u kreiranju test slučajeva, njihovoj automatizaciji, izvršavanju i izveštavanju

Planiranje testiranja



Planiranje



Ako bih imao devet sati da isečem drvo, prvih šest sati bih proveo oštreći sekiru.

Abraham Lincoln

Planiranje testiranja

- Podrazumeva kreiranje različitih planova kao pripremu za proces testiranja
- Planovi predviđaju
  - šta se testira
  - kako se vrši testiranje,
  - kada se vrši testiranje,
  - ko vrši testiranje
  - koje su sve dodatne aktivnosti potrebne da bi testiranje bilo izvršeno i kako se obezbeđuju

Plan obezbeđenja kvaliteta

- Planira resurse i procedure koji treba da obezbede kvalitet finalnog proizvoda
- Testiranje je jedan način obezbeđenja kvaliteta
- Stoga testiranje mora biti uključeno u generalni plan obezbeđenja kvaliteta
- Osim toga uključuje
  - primenu standarda i konvencija
  - korišćenje odgovarajućih metrika za evaluaciju kvaliteta
  - obuku korisnika
  - upravljanje rizikom
  - ...

Obezbeđenje kvaliteta

- Obezbeđenje kvaliteta se vrši primenom standarda, pravila, zakonskih obaveza
  - Kompanijski standardi
    - interne odredbe, procedure i smernice
    - npr. dokument o upravljanju kvalitetom, šabloni za test dokumentaciju, konvencije programskog koda
  - Dobre prakse
    - široko prihvaćene metode rada u određenoj oblasti
  - Standardi upravljanja kvalitetom
    - generalni standardi koji se odnose na različite industrijske grane
    - specificiraju minimalne zahteve kvaliteta
    - ne bave se metodama implementacije kojima se kvalitet dostiže
  - Standardi za specifičnu industrijsku granu
    - svaka industrija ima svoje standarde kvaliteta
  - Standardi za testiranje softvera
    - odnose se na bilo koji tip softvera
    - bave se dokumentacijom i procesom testiranja
    - npr. IEEE 829, IEEE 1028, ISO 29119

Test plan

- Dokument koji sadrži podatke koji se tiču planiranja testiranja aplikacije
- Podaci se tiču sledećih aktivnosti planiranja
  - izbor generalnog pristupa i strategije testiranja
  - izbor test okruženja i načina automatizacije testova
  - izbor nivoa testiranja i veza između njih
  - integracija testiranja sa drugim aktivnostima u okviru projekta
  - izbor načina evaluacije rezultata testiranja
  - izbor metrike za kriterijum završetka testiranja

Test plan

- Aktivnosti planiranja (nastavak sa prethodnog slajda)
  - izbor načina dokumentovanja testiranja i izbor šablona dokumenata
  - izbor šta se testira
  - ko i kada vrši testiranje
  - odluka o intenzitetu testiranja
  - izračunavanje napora i troškova potrebnih/utrošenih za testiranje

Test plan prema IEEE 829-2008

- Predviđa
  - master test plan
    - sadrži generalni plan testiranja za ceo projekat
    - za svaku fazu životnog ciklusa testiranja predviđa konkretne aktivnosti, resurse i učesnike
  - level test plan
    - plan testiranja za određeni nivo testiranja
    - tako imamo *component test plan, integration test plan, system test plan*, ...
    - za svaki nivo testiranja se definiše
      - pristup, resursi i raspored aktivnosti
      - šta će biti testirano
      - konkretne aktivnosti kojima će se vršiti testiranje
      - odgovorne osobe za svaku aktivnosti

Master test plan prema IEEE 829-2008

Master Test Plan Outline (full example)

1. Introduction

1.1. Document identifier

1.2. Scope

1.3. References

1.4. System overview and key features

1.5. Test overview

1.5.1 Organization

1.5.2 Master test schedule

1.5.3 Integrity level schema

1.5.4 Resources summary

1.5.5 Responsibilities

1.5.6 Tools, techniques, methods, and metrics

2. Details of the Master Test Plan

2.1. Test processes including definition of test levels

2.1.1 Process: Management

2.1.1.1 Activity: Management of test effort

2.1.2 Process: Acquisition

2.1.2.1: Activity: Acquisition support test

2.1.3 Process: Supply

2.1.3.1 Activity: Planning test

2.1.4 Process: Development

2.1.4.1 Activity: Concept

2.1.4.2 Activity: Requirements

Master test plan prema IEEE 829-2008

2.1.4.3 Activity: Design

2.1.4.4 Activity: Implementation

2.1.4.5 Activity: Test

2.1.4.6 Activity: Installation/checkout

2.1.5 Process: Operation

2.1.5.1 Activity: Operational test

2.1.6 Process: Maintenance

2.1.6.1 Activity: Maintenance test

2.2. Test documentation requirements

2.3. Test administration requirements

2.4. Test reporting requirements

3. General

3.1. Glossary

3.2. Document change procedures and history

Level test plan prema IEEE 829-2008

1. Introduction

1.1. Document identifier

1.2. Scope

1.3. References

1.4. Level in the overall sequence

1.5. Test classes and overall test conditions

2. Details for this level of test plan

2.1 Test items and their identifiers

2.2 Test Traceability Matrix

2.3 Features to be tested

2.4 Features not to be tested

2.5 Approach

2.6 Item pass/fail criteria

2.7 Suspension criteria and resumption requirements

2.8 Test deliverables

Level test plan prema IEEE 829-2008

3. Test management

3.1 Planned activities and tasks; test progression

3.2 Environment/infrastructure

3.3 Responsibilities and authority

3.4 Interfaces among the parties involved

3.5 Resources and their allocation

3.6 Training

3.7 Schedules, estimates, and costs

3.8 Risk(s) and contingency(s)

4. General

4.1 Quality assurance procedures

4.2 Metrics

4.3 Test coverage

4.4 Glossary

4.5 Document change procedures and history

Prioriteti testova

- Nekada vreme i budžet ne dozvoljavaju kreiranje ili izvršavanje svih planiranih testova ili nivoa testiranja
- Tada se mora napraviti izbor testova koji se realizuju
  - moraju se uspostaviti prioriteti testova
- Pravilo uspostavljanja prioriteta
  - ako se testiranje prevremeno zaustavi, moramo biti sigurni da su postignuti najbolji mogući rezultati testiranja obzirom na dužinu trajanja testiranja
- To znači da prvo treba realizovati važnije testove
  - kako odrediti koji je test važniji?

Kriterijumi izbora prioriteta testa

- Određuje se na osnovu funkcionalnosti koju test testira
- Sledeće osobine testa se uzimaju u obzir
  - učestalost korišćenja i verovatnoća otkaza funkcionalnosti
    - ako često korišćena funkcionalnost sadrži nedostatak, verovatno će taj nedostatak izazvati otkaz sistema
  - Rizik otkaza funkcionalnosti
    - rizik se izračunava kao proizvod štete koju otkaz izaziva i verovatnoće da se otkaz desi

Kriterijumi izbora prioriteta testa

- Karakteristike testa za određivanje prioriteta (nastavak sa prethodnog slajda)
  - Vidljivost otkaza za krajnjeg korisnika
    - nedostaci koji se tiču grafičkog interfejsa ili onemogućuju korišćenje sistema imaju veći prioritet u otklanjanju

> *Ma neće to niko ni primetiti.*
> Neimenovani programer

  - Važnost zahteva
    - naručilac pridaje različit značaj pojedinačnim zahtevima
    - prioritet ima testiranje funkcionalnosti koje naručilac vidi kao najznačajnije
      - to ne moraju uvek biti i objektivno najznačajnije funkcije

Kriterijumi izbora prioriteta testa

- Karakteristike testa za određivanje prioriteta (nastavak sa prethodnog slajda)
  - važnost nefunkcionalnog zahteva
    - ako test testira nefunkcionalni zahtev
    - nemaju svi nefunkcionalni zahtevi za naručioca istu važnost
    - npr. za neku aplikaciju koju koristi ograničeni broj korisnika zahtevi za dobrim performansama mogu biti važniji od bezbednosti aplikacije
  - važnost testa iz perspektive razvoja aplikacije
    - programer analizira kakve posledice nedostatak u testiranoj komponenti ima na kompletan razvoj aplikacije

Kriterijumi izbora prioriteta testa

- Karakteristike testa za određivanje prioriteta (nastavak sa prethodnog slajda)
  - Kompleksnost testirane komponente
    - kompleksni delovi programa trebaju biti intenzivnije testirani
      - potencijalno kriju više nedostataka
    - voditi računa i da naizgled jednostavni delovi programa mogu imati nedostatke zbog manje pažnje s kojima su razvijani
    - iskustveno proceniti broj potencijalnih nedostataka
  - Količina otkaza u komponenti
    - obično se u komponenti u kojoj se pronađe puno nedostataka nalazi još dosta neotkrivenih
    - ovakve komponente trebaju biti među prioritetima u testiranju

Kriterijum početka testiranja

- Određeni preduslovi moraju biti ispunjeni da bi se izvršilo testiranje
  - neracionalno je trošiti resurse na testiranje nespremnog proizvoda
- Kriterijumi početka testiranja
  - Test okruženje mora biti spremno
  - Test alati su spremni za korišćenje
  - Test objekti postoje i nalaze se u dovoljno zrelom stanju razvoja da ih ima smisla testirati
  - Ulazni test podaci su dostupni

Kriterijum završetka testiranja

- Kriterijum završetka je neophodan da se testiranje ne bi završilo prerano
  - ne sme finalni proizvod da bude „nedovoljno“ testiran
    - uvek postoji ova opasnost zbog ograničenog vremena i resursa
  - koliko je „dovoljno“ određuje ovaj kriterijum

Kriterijum završetka testiranja

- Mogući kriterijumi završetka
  - dostignuta tražena pokrivenost
    - pokrivenost koda, pokrivenost zahteva ili količina izvršenih testova
  - kvalitet proizvoda
    - učestalost otkaza ili ozbiljnost nedostataka
  - preostali rizik
    - količina neizvršenih testova, neispravljenih nedostataka, nepokrivenih nedostataka ili koda
  - ekonomska ograničenja
    - dozvoljeni troškovi, rokovi isporuke, kriterijumi tržišta

Ekonomski aspekti



Ekonomski aspekti testiranja

- Testiranje podrazumeva određene troškove
- Neophodno je pratiti troškove u odnosu na budžet projekta i koristi koje testiranje donosi
- Pri analizi troškova testiranja mora se uzeti u obzir i koliki bi bio trošak otkaza izazvan netestiranim nedostatkom
  - porede se troškovi nedostataka sa troškovima testiranja

Troškovi nedostataka

- Izbacivanje testiranja bi moglo izazvati sledeće troškove
  - Direktni troškovi nedostatka
    - troškovi koje korisnik trpi zbog greške u funkcionisanju sistema
      - npr. izgubljena dobit zbog pogrešnog izračunavanja, gubitak podataka zbog neispravnog skladištenja, oštećenje opreme, ...
  - Indirektni troškovi nedostatka
    - troškovi koje klijent trpi zbog nezadovoljstva korisnika
      - pad prodaje proizvoda
    - povećani troškovi korisničke podrške
    - pravni troškovi
      - gubitak licence zbog neusklađenosti sa standardima

Troškovi nedostataka

- Troškovi nedovoljnog testiranja (nastavak sa prethodnog slajda)
  - troškovi ispravke nedostatka
    - resursi potrebni za analizu otkaza, ispravku, ponovno testiranje, reinstalaciju, ponovnu obuku
    - trpe i drugi projekti jer se resursi moraju angažovati na ispravku neispravnog proizvoda

Troškovi nedostataka

- Kakav i koliki trošak nedostatak izaziva zavisi od mnogo faktora
  - tipa nedostatka, tipa aplikacije, trenutka u kojem je nedostatak otkriven
- Što je nedostatak kasnije otkriven, troškovi su veći
  - nedostatak napravljen u ranoj fazi razvoja (npr. u specifikaciji zahteva) povlači nove nedostatke u daljim fazama razvoja
  - kasno otkriven nedostatak zahteva modifikaciju mnogih delova aplikacije i ponavljanje različitih faza razvoja
    - čak i trivijalan nedostatak proizvodi velike troškove ukoliko se nedostatak otkrije tek kad je aplikacija u produkciji
  - okvirno, sa svakim nivoom testiranja se udvostručuje trošak u odnosu na trošak pronalaženja nedostatka u prethodnom nivou

Douglas MacArthur

Supreme Commander for the Allied Powers.

C.W. Nimitz

United States Representative

徐永昌

Republic of China Representative

Bruce Fraser

United Kingdom Representative

Jacob Malik

Union of Soviet Socialist Republics Representative

Lavrentiy Alexandrovich Govorov

Commonwealth of Australia Representative

Dominion of Canada Representative

Jacques Leclerc

Provisional Government of the French Republic Representative

DOMINION OF CANADA REPRESENTATIVE

PROVISIONAL GOVERNMENT OF THE FRENCH REPUBLIC REPRESENTATIVE

Kingdom of the Netherlands Representative

KINGDOM OF THE NETHERLANDS REPRESENTATIVE

Dominion of New Zealand Representative

DOMINION OF NEW ZEALAND REPRESENTATIVE

Troškovi testiranja

- Podrazumeva sve troškove koje aktivnosti testiranja zahtevaju
  - priprema, dizajniranje, implementacija, izveštavanje, ...
- Troškove unosi korišćenje različitih resursa
  - vreme, ljudi, oprema
  - dobro planiranje aktivnosti testiranja bi trebalo da smanji troškove koje testiranje unosi

Troškovi testiranja

- Faktori koji utiču na troškove testiranja
  - zrelost procesa razvoja softvera
    - stabilnost organizacije u kojoj se sistem razvija
    - razvijenost poslovnih procesa u organizaciji
    - kvalitet programera (broj grešaka koje produkuju)
    - vremenski okvir projekta
    - kvalitet planova testiranja
    - stepen razvijenosti procesa testiranja (organizovanost upravljanja testiranjem)

Troškovi testiranja

- Faktori koji utiču na troškove testiranja (nastavak sa prethodnog slajda)
  - kvalitet i mogućnost testiranja softvera
    - broj i raspoređenost nedostataka u programu
    - količina i kvalitet dokumentacije koja se koristi kao osnova testiranja
    - veličina, tip i način implementacije softvera
      - neke tehnologije podstiču pisanje koda koji se može lakše testirati
    - kompleksnost implementacije softvera

Troškovi testiranja

- Faktori koji utiču na troškove testiranja (nastavak sa prethodnog slajda)
  - infrastruktura za testiranje
    - dostupnost alata za testiranje
    - kvalitet test okruženja
    - tehnologija koja se koristi za testiranje
    - postojanje i dostupnost standarda i procedura za testiranje
  - kvalifikovanost kadrova
    - iskustvo koje testeri imaju u testiranju
    - iskustvo koje testeri imaju sa test alatima, okruženjem i tehnologijom koja se koristi u testiranju
    - poznavanje test objekata koje testiraju
    - saradnja između testera, programera, menadžera i klijenta

Troškovi testiranja

- Faktori koji utiču na troškove testiranja (nastavak sa prethodnog slajda)
  - zahtevani kvalitet
    - zahtevana pokrivenost
    - zahtevana pouzdanost
      - koja količina nedostataka preostalih nakon testiranja se toleriše
    - zahtevani nivo nefunkcionalnih zahteva
      - bezbednost, performanse, ...
      - od toga zavisi da li je određeni tip testiranja uopšte potrebno uvoditi
      - svaki tip testiranja zahteva nove troškove, uvođenje novih specijalista u projekat, ...
    - zahtevana količina dokumentacije

Troškovi testiranja

- Faktori koji utiču na troškove testiranja (nastavak sa prethodnog slajda)
  - pristup testiranju
    - ciljevi testiranja
      - zavise od zahtevanog kvaliteta softvera
    - način testiranja
      - nivoi i tipovi testiranja
    - raspored testiranja
      - organizacija aktivnosti testiranja u okviru kompletnog životnog ciklusa softvera

Uticaj na troškove testiranja

- Samo na neke od pomenutih troškova se može uticati dobrim upravljanjem
  - neki kao što je npr. tip projekta su određeni projektnim zadatkom
- Troškovi na koje se može uticati
  - zrelost procesa razvoja softvera
    - može se samo dugoročno rešavati
    - u datom trenutku se uvek mora prihvatiti trenutna zrelost organizacije u kojoj se projekat razvija
  - mogućnost testiranja softvera
    - zavisi od načina implementacije i procesa razvoja
    - podsticanjem dobrih praksi razvoja softvera, test menadžer može delimično da utiče na to da dobije softver koji je prilagođeniji za testiranje
    - neophodno je od samog početka razvoja razmišljati i o testiranju

Uticaj na troškove testiranja

- Troškovi na koje se može uticati (nastavak sa prethodnog slajda)
  - infrastruktura za testiranje
    - određena je okolnostima u kojima se proces razvija, ali se jednim delom može uticati na to da bude odgovarajuća za efikasan proces testiranja
    - u toku razvoja potrebno je infrastrukturu dalje unapređivati da bi se uštedelo vreme i novac za testiranje
  - kvalifikovanost kadrova
    - određeno okolnostima, ali se može menjati promenom kadrova ili obukom postojećih

Uticaj na troškove testiranja

- Troškovi na koje se može uticati (nastavak sa prethodnog slajda)
  - zahtevani kvalitet
    - zadat je od strane naručioca
    - kod sopstvenih projekata određen namenom aplikacije i odlukama višeg menadžmenta na koje test menadžment obično nema uticaj
    - moguće je uticati na prioritete ispunjavanja zahteva
  - pristup testiranju
    - ovo je jedina stavka na koju se uglavnom može potpuno uticati
    - potrebno je izabrati efikasan pristup koji će smanjiti troškove

Izračunavanje napora utrošenog u testiranje

- Pre izbora i alokacije resursa potrebno je napraviti procenu o potrebnom naporu koji će biti utrošen u testiranje
- Dva pristupa proceni
  1. napraviti spisak svih zadataka vezanih za testiranje i zahtevati od izvršioca ili stručnjaka da procene obim zadatka
  2. proceniti napor na osnovu prethodnih sličnih projekata
- Nepisano pravilo koje može da pomogne
  - zadaci vezani za testiranje u tipičnoj poslovnoj aplikaciji troše oko 50% ukupnih resursa alociranih za projekat

Izbor strategije testiranja



Test strategija ili pristup testiranju

- Definiše ciljeve testiranja i načine kako da budu ostvaren
  - time su definisani i napor i troškovi testiranja
- Cilj je izabrati strategiju koja optimizuje odnos troškova testiranja i mogućih troškova otkaza, kao i da minimizuje rizike
- Podrazumeva se da trošak testiranja treba da bude manji od troška mogućeg otkaza
  - retko se raspolaže svim podacima koji utiču na trošak otkaza
  - teško je egzaktno izračunati koliki bi trošak otkaza bio
  - zato se često procena vrši intuitivno umesto racionalnog izračunavanja

Preventivni vs reaktivni pristup

- Ova dva pristupa razlikuju se po trenutku kada se testeri uvode u projekat
- Preventivni pristup podrazumeva uključivanje testera od samog početka
  - testiranje je uklopljeno u proces razvoja
  - moguće je optimizovati testiranje i smanjiti troškove
  - praćenje nekog od standardnih modela razvoja softvera bi trebalo da ovo obezbedi
  - važno je definisati test specifikaciju i posvetiti se planiranju testiranja

Preventivni vs reaktivni pristup

- Reaktivni pristup testiranju se primenjuje kada razvoj projekta nije od početka uzeo u obzir testiranje
  - testeri su kasno uvedeni
  - testiranje počinje tek nakon što je sistem završen ili razvijen značajan deo
  - aktivnosti testiranja nisu bile uklopljene u životni ciklus razvoja softvera
  - nije obavljeno pravovremeno planiranje testiranja
  - test menadžer mora naći način da naknadno integriše testiranje u takav projekat
    - npr. *exploratory testing* kao način naknadnog izučavanja i testiranja test objekta

Preventivni vs reaktivni pristup

- Treba birati preventivni pristup kad god je moguće
  - testiranje mora da počne što ranije
  - testirati treba kontinuirano kroz sve faze razvoja
  - kod kritičnih aplikacija je takav pristup oduvek bio obavezan
  - ovakav pristup postaje standard kod sve većeg broja aplikacija
    - kod poslovnih informacionih sistema je danas testiranje uključeno od početka razvoja

Analitički vs iskustveni pristup

- Analitički pristup
  - planiranje je zasnovano na podacima i analizi primenom neke egzaktne metode
  - parametri od kojih testiranje zavisi se kvantifikuju
  - pristup testiranju se bira tako da se optimizuju različiti parametri
- Iskustveni pristup
  - planiranje je zasnovano na iskustvu stručnjaka
    - internih ili eksternih
  - takođe, koriste se nepisana pravila, opšteprihvaćene procene i slično
  - razlog ovakvog pristupa je što je teško tako komplikovan proces egzaktno opisati i evaluirati

Pristupi koji se koriste u praksi

- Testiranje zasnovano na modelu
  - kreira se apstraktni model sistema za dizajn test slučajeva, definisanje kriterijum završetka i merenje pokrivenosti sistema testovima
  - jedan primer je testiranje prelaza stanja gde su dijagrami stanja model
- Statističko testiranje
  - test pristup se određuje na osnovu statističke analize distribucije nedostataka, učestalosti otkaza itd.

Pristupi koji se koriste u praksi

- Testiranje zasnovano na riziku
  - koristi analizu rizika za izbor tehnike testiranja
- Pristupi usklađeni sa procesom ili standardom
  - koristi pravila, preporuke i standarde iz oblasti razvoja softvera, kao i dobre prakse u konkretnoj tehnologiji u kojoj se softver razvija
- Pristupi usmereni na ponovno iskorišćenje
  - bira se pristup koji će omogućiti ponovno korišćenje postojećeg test okruženja

Pristupi koji se koriste u praksi

- Pristupi zasnovani na listama
  - koristi se lista otkaza i nedostataka iz prethodnih faza razvoja
- Pristupi zasnovani na stručnjacima
  - koristi se iskustvo stručnjaka uključenih u projekat
  - prepušta se njihovoj proceni izbor tehnike testiranja
  - pri izboru oni implicitno ili eksplicitno uzimaju u obzir različite faktore
- Uglavnom je pristup kombinacija različitih tipova pristupa

Testiranje i rizik

- Potencijalni rizik je jedan od najboljih kriterijuma za izbor načina testiranja
- $rizik = šteta\ x\ verovatnoća$
- Razlikujemo
  - rizike projekta
    - rizici koji ugrožavaju završetak projekta
  - rizike finalnog proizvoda
    - rizici koji su posledica problema sa isporučenim proizvodom

Rizici projekta

- Mogući rizici
  - neodgovarajuća podrška za projekat
    - npr. partnerska firma koja ne ispunjava ugovor
  - organizacijski rizici
    - nedostatak internih resursa
      - nedovoljan broj ili kvalitet kadrova
    - ljudski faktor
      - loši međusobni odnosi ili neodgovarajući psihološki profil zaposlenih
    - organizacijski problemi
      - loša struktura i poslovni procesi organizacije u kojoj se softver razvija

Rizici projekta

- Rizici projekta (nastavak sa prethodnog slajda)
  - tehnički problemi
    - pogrešni, nekompletni, nedovršeni, kontradiktorni zahtevi
    - nedostaci tehnologije
      - obično se otkriju u odmakloj fazi projekta
      - često kada je tehnologija nova za projektni tim
    - problemi sa hardverskom i softverskom infrastrukturom

Rizici finalnog proizvoda

- Mogući rizici
  - neodgovarajući kvalitet
    - funkcionalni ili nefunkcionalni zahtevi nisu zadovoljeni
    - proizvod ne ispunjava namenu
    - kvalitet podataka je neodgovarajući
    - proizvod ugrožava sigurnost opreme i ljudi koji ga koriste
  - organizacioni i psihološki faktori kod naručioca
    - neodgovarajuća podrška za uvođenje aplikacije može da ugrozi korišćenje finalnog proizvoda
    - zaposleni često imaju otpor da koriste nov proizvod

Upravljanje rizikom

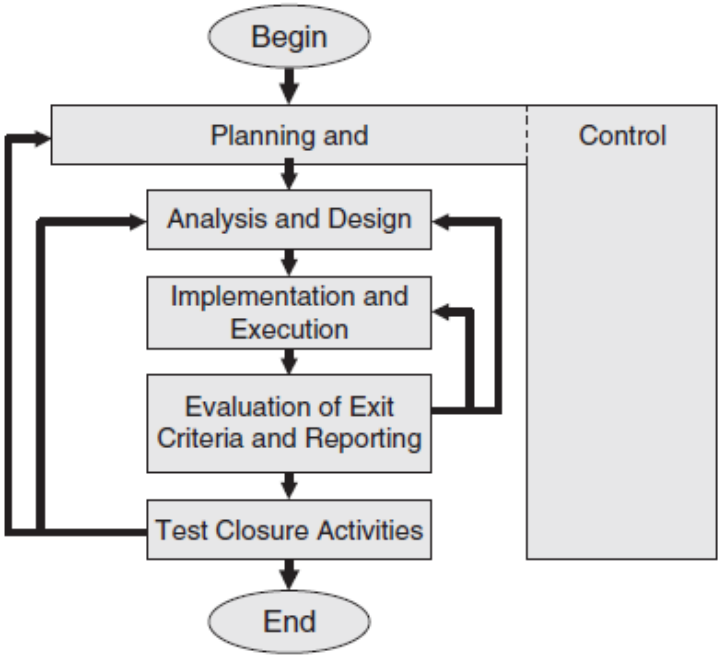
- Standardi obezbeđenja kvaliteta predviđaju sledeće akcije
  - redovno identifikovanje mogućih rizika
  - uspostavljanje prioriteta među rizicima
  - izvođenje akcija koje smanjuju ili otklanjaju rizike
    - testiranje je aktivnost koja **smanjuje** rizik
- Testiranje zasnovano na riziku na osnovu procenjenog rizika bira
  - tehniku testiranja,
  - obim i intenzitet testiranja,
  - prioritet test slučajeva

Upravljanje ciklusom testiranja



Ciklus testiranja

- Za svaki nivo testiranja prolazi se kroz kompletan ciklus testiranja
- Nakon svakog ciklusa potrebna je izmena u kodu
- Nakon izmena potrebno je ponoviti testove
- Upravljanje testiranjem vodi računa o inicijaciji, nadzoru i upravljanju ovim ciklusom



Planiranje ciklusa testiranja

- Nakon generalnog plana za kompletno testiranje, pravi se plan za konkretan ciklus testiranja
- Stavke koje treba uzeti u obzir pri pravljenju plana
  - Stanje projekta
    - treba proveriti kako se trenutno stanje projekta odnosi prema stanju koje je predviđeno kada je pravljen generalni plan
    - aktivnosti testiranja se moraju prilagoditi trenutnom stanju
  - Rezultati testova
    - ako je reč o novoj iteraciji određenog ciklusa testiranja treba uzeti u obzir rezultate iz prethodne iteracije
    - možda je potrebno dodatno testirati funkcionalnost koja je ispravljena

Planiranje ciklusa testiranja

- Stavke koje treba uzeti u obzir pri pravljenju plana (nastavak sa prethodnog slajda)
  - Resursi
    - plan se mora uklopiti u plan celog projekta
    - voditi računa da testiranje može da zahteva iste resurse kao i neke druge aktivnosti u projektu
    - takođe, potrebno se uklopiti u raspored resursa koji nema veze sa testiranjem
      - npr. godišnji odmori radnika
- Na osnovu pomenutih parametara određuje se konkretan raspored testiranja koji uključuje konkretne test slučajeve, kao i ko će ih i kada izvršavati

Nadzor ciklusa testiranja

- Neophodno je nadgledati i meriti rezultate testa
  - potrebna je pouzdana i jednostavna metrika
- Mogući pristupi
  - metrike zasnovane na otkazima
    - broj otkaza po test objektu
    - otkazi se klasifikuju prema tipu i statusu
    - svaki otkaz ukazuje i na liniju koda u kojoj se desio
  - metrike zasnovane na test slučajevima
    - broj test slučajeva u određenom stanju
      - izvršen, otkazan, uspešan, neuspešan

Nadzor ciklusa testiranja

- Mogući pristupi
  - metrike zasnovane na test objektu
    - pokrivenost koda
    - varijante test objekta
      - npr. rezultati za različite platforme
  - metrike zasnovane na troškovima
    - dosadašnji troškovi testiranja
    - trošak sledećeg ciklusa u poređenju sa očekivanom koristi

Nadzor ciklusa testiranja

- Izmereni rezultati testiranja se navode u test izveštaju
- Izveštaj sadrži informacije o statusu testova:
  - generalni podaci
    - koji test objekti su testirani na kom nivou i u kojem test ciklusu
  - napredak
    - planirani, izvršeni i otkazani testovi
  - status grešaka
    - nova, u toku ispravljanja, ispravljena
  - rizici
    - novi, izmenjeni, poznati
  - plan narednog ciklusa testiranja
  - ocena
    - ocena zrelosti test objekta i procena mogućnosti za stavljanje u produkciju
    - ovo je i kriterijum završetka testiranja

Upravljanje ciklusom testiranja

- Odnosi se na prilagođavanje aktivnosti u ciklusu trenutnim okolnostima projekta
  - vremenski raspored faza može zahtevati prilagođavanje
  - može biti potrebno uvesti dodatne resurse
  - nekad se izvršavaju samo prioritetni testovi zbog nedostatka vremena
  - s druge strane, nekad se planirano vreme produžava zbog detaljnije provere

Upravljanje greškama



Upravljanje greškama

- Mora da postoji definisana procedura opisa i prijave grešaka
  - ovakva procedura obezbeđuje efikasno pronalaženje i otklanjanje grešaka na osnovu otkaza
- Test log treba da sadrži sve informacije o rezultatima testiranja

Test log

- Nakon izvršavanja testova, analizira se test log
  - porede se očekivani sa stvarnim rezultatima
  - kod automatizovanih testova ovo uradi alat

- Nakon toga se greška prijavljuje
  - za stavke test loga kod kojih očekivano ponašanje odstupa od stvarnog
  - mora se proveriti da je uzrok test objekat, a ne greška u testu

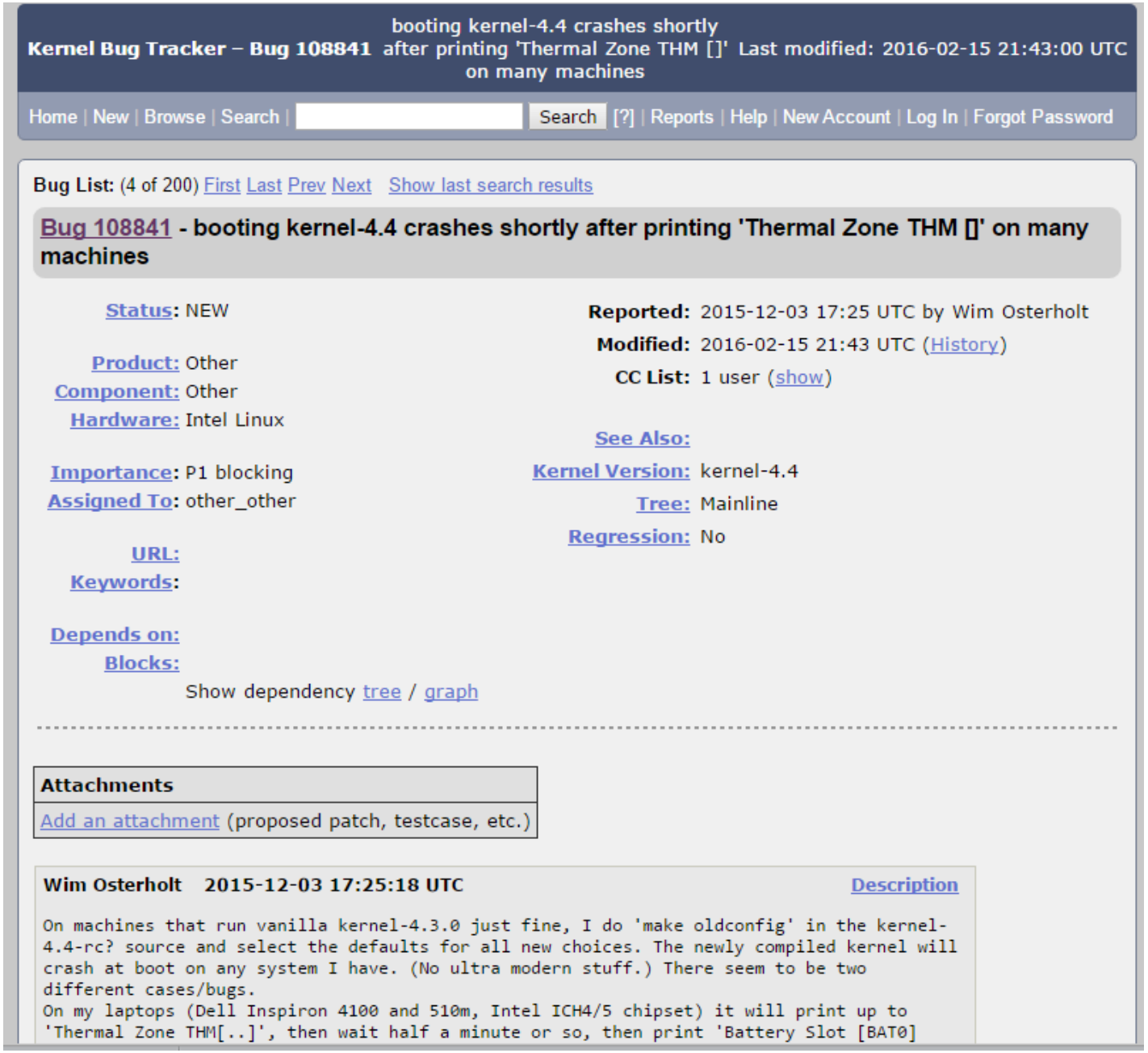
Prijavljivanje grešaka

- Uspostavlja se centralizovana baza grešaka za projekat
  - kroz specijalizovane softverske alate
  - često integrisano sa upravljanjem drugim projektnim aktivnostima
    - GitLab, Redmine, ...
- Registruju se korisnici koji mogu da prijave greške
  - članovi razvojnog tima i korisnici

Prijavljivanje grešaka

- Svaka greška se prijavljuje u dogovorenom formatu
  - opis greške
    - sadrži i opis kako greška može biti reprodukovana
  - test okruženje u kojem se greška desila
  - prioritet
  - tip greške
  - status
  - podaci o testeru
  - komentar
- Korisnike je teže obučiti da prijavljuju greške po dogovorenoj proceduri

	Attribute	Meaning
Identification	Id / Number	Unique identifier/number for each report
	Test object	Identifier or name of the test object
	Version	Identification of the exact version of the test object
	Platform	Identification of the HW/SW platform or the test environment where the problem occurs
	Reporting person	Identification of the reporting tester (possibly with test level)
	Responsible developer	Name of the developer or the team responsible for the test object
	Reporting date	Date and possibly time when the problem was observed
Classification	Status	The current state (and complete history) of processing for the report (section 6.6.4)
	Severity	Classification of the severity of the problem (section 6.6.3)
	Priority	Classification of the priority of correction (section 6.6.3)
	Requirement	Pointer to the (customer-) requirements which are not fulfilled due to the problem
	Problem source	The project phase, where the defect was introduced (analysis, design, programming); useful for planning process improvement measures
Problem description	Test case	Description of the test case (name, number) or the steps necessary to reproduce the problem
	Problem description	Description of the problem or failure that occurred; expected vs. actual observed results or behavior
	Comments	List of comments on the report from developers and other staff involved
	Defect correction	Description of the changes made to correct the defect
	References	Reference to other related reports



booting kernel-4.4 crashes shortly after printing 'Thermal Zone THM []' on many machines

Kernel Bug Tracker – Bug 108841 Last modified: 2016-02-15 21:43:00 UTC

Home | New | Browse | Search | Search [?] | Reports | Help | New Account | Log In | Forgot Password

Bug List: (4 of 200) First Last Prev Next Show last search results

Bug 108841 - booting kernel-4.4 crashes shortly after printing 'Thermal Zone THM []' on many machines

Status: NEW

Product: Other
Component: Other
Hardware: Intel Linux

Importance: P1 blocking
Assigned To: other_other

URL:
Keywords:

Depends on:
Blocks:

Show dependency tree / graph

Reported: 2015-12-03 17:25 UTC by Wim Osterholt
Modified: 2016-02-15 21:43 UTC (History)
CC List: 1 user (show)

See Also:
Kernel Version: kernel-4.4
Tree: Mainline
Regression: No

Attachments

Add an attachment (proposed patch, testcase, etc.)

Wim Osterholt 2015-12-03 17:25:18 UTC Description

On machines that run vanilla kernel-4.3.0 just fine, I do 'make oldconfig' in the kernel-4.4-rc? source and select the defaults for all new choices. The newly compiled kernel will crash at boot on any system I have. (No ultra modern stuff.) There seem to be two different cases/bugs.
On my laptops (Dell Inspiron 4100 and 510m, Intel ICH4/5 chipset) it will print up to 'Thermal Zone THM[..]', then wait half a minute or so, then print 'Battery Slot [BAT0]

Prijavljivanje grešaka

- Prijava greške povlači dalje akcije
  - komunikaciju programera i testera
  - moguće razjašnjenje ili proveru prijavljene greške
  - ispravku
  - prijavu i objašnjenje ispravke
  - nakon ovoga tester može da proveri da li je ispravka otklonila grešku

Klasifikacija grešaka

- Važna stvar u odluci kako će greška biti tretirana je ozbiljnost greške
  - odnosi se na to u kojoj meri greška ugrožava rad programa
  - npr. greška koja prekida rad sistema je ozbiljnija od neke vizuelne greške u grafičkom interfejsu

Klasifikacija grešaka

Klasa	Opis
1 – Fatalna	• Rad sistema se prekida, moguće sa gubitkom podataka • Test objekat ne može biti pušten u produkciju sa ovakvom greškom
2 – Veoma ozbiljna	• Važna funkcionalnost ne radi ispravno • Zahtev nije zadovoljen • Neprihvatljivo za većinu uključenih u projekat (menadžment, naručilac, korisnici) • Test objekat može biti korišćen samo uz značajna ograničenja (komplikovano i skupo zaobilaženje greške)
3 – Ozbiljna	• Odstupanje od ispravne funkcionalnosti • Zahtev nepotpuno ili netačno implementiran • Neprihvatljivo za neke od uključenih u projekat
4 – Umerena	• Manje odstupanje od zahtevanog ponašanja • Sistem se može koristiti bez ograničenja
5 – Neznatna	• Neznatna greška u realizaciji • npr. slovna greška

Prioritet greške

- Pored klasifikacije greške, potrebno je odrediti koliko brzo je grešku neophodno otkloniti
- Zavisi od klase greške, ali i drugih faktora
  - težina otklanjanja
  - uticaj greške na druge delove projekta
  - raspoloživost odgovornih programera

Proritet greške

Prioritet	Opis
1 – Odmah	• Korišćenje sistema ili dalji razvoj projekta je blokiran zbog ove greške • Greška se mora ispraviti bez odlaganja
2 – Za sledeću verziju	• Ispravka će biti ugrađena u sledeću redovnu verziju proizvoda koja se postavi kod korisnika
3 – Po prilici	• Ispravka će biti urađena kada je taj deo sistema inače predviđen za izmenu
4 – Otvoren	• Još uvek nije određeno kada će ispravka biti urađena

Trend prijave grešaka

- Trend prijave grešaka ukazuje na trenutno stanje projekta
- Koliko je projekat blizu finalnog proizvoda se može zaključiti iz broja i tipa grešaka
- Potrebno je proceniti koliko se očekuje grešaka u narednom periodu
- Treba analizirati i raspoređenost grešaka po test objektu
  - za test objekat sa puno grešaka se mogu očekivati i buduće greške
  - za test objekat sa malo grešaka treba proveriti da li su testovi odgovarajući

Status greške

- Iako testeri ne vrše ispravku greške, upravljanje testiranjem treba da obezbedi praćenje njenog statusa
- Svaka greška prolazi kroz niz predefinisanih statusa

Statusi greške

Status (ko postavlja)	Opis
Novi (tester)	• Greška je tek evidentirana
Otvoren (test menadžer)	• Test menadžer analizira evidentirane greške • Na osnovu opisa određuje da li je prijava smislena • Ako jeste, zahteva se ispravka greške od određenog programera i postavlja u ovaj status
Odbijen (test menadžer)	• Već prijavljena ili neodgovarajuća prijava • Ispravka se ne zahteva, prijava se odbija
Analiza (programer)	• Programer analizira grešku čija je ispravka zatražena • Određuje uzrok, način rešavanja, obim posla
Posmatranje (programer)	• Prijavljena greška ne može biti reprodukovana • Potrebna dodatna pojašnjenja

Statusi greške (nastavak)

Status (ko postavlja)	Opis
Ispravka (menadžer projekta)	• Na osnovu analize menadžer projekta određuje da li, kada i ko treba raditi ispravku • Programer vrši ispravku i postavlja komentar o izvršenim izmenama
Test (programer)	• Označava da je programer izvršio ispravku i da izmena treba da bude testirana
Zatvoren (tester)	• U narednom ciklusu testiranja testiraju se ispravke koje su u statusu *test* • Ponavlja se test slučaj kojim je greška otkrivena • Ako tester utvrdi da je ispravka otklonila grešku, postavlja status u *zatvoren* • samo tester (a ne programer) može da zatvori prijavu
Neuspešan (tester)	• Ako verifikacija ispravke pokaže da ispravka nije otklonila grešku, status je *neuspešan*

Dijagram prelaza statusa greške

